

logger

The main purpose is to **understand what your app is doing** — especially when things go wrong.

Without a logger, you probably use `print()`:

```
print("user logged in")
print("something broke")
```

This works locally but falls apart in real apps because:

- no timestamps
- no severity levels
- can't filter noise from important messages
- disappears when the terminal closes
- can't write to a file automatically

With a logger, you get much more:

```
2026-05-21 10:32:01 | INFO | User alice logged in
2026-05-21 10:32:05 | ERROR | Payment failed for order #9921
2026-05-21 10:32:06 | ERROR | DB connection timeout
```

Purpose

1. Debugging — trace what happened before a crash

```
logger.debug(f"Calling API with params: {params}")
logger.debug(f"Got response: {response}")
```

2. Monitoring in production — know your app is healthy

```
logger.info("Scheduled job completed: 1,200 records processed")
```

3. Alerting on errors — catch failures fast

```
logger.error("Stripe API unreachable") # → triggers an alert to your team
```

4. Audit trail — record who did what

```
logger.info(f"User {user_id} deleted record {record_id}")
```

5. Performance tracking

```
logger.info(f"Query took {elapsed_ms}ms") # spot slow queries
```